

2018-2019  
Conduite de Projets Objet  
Contrôle de Versions de  
Code Source  
Git

Anne-Gwenn Bosser

# Plan du cours aujourd'hui – Objectifs pédagogiques

## Sujets abordés aujourd'hui:

- Comment développer à plusieurs sur le même projet sans avoir de problèmes quand on met tout ensemble?
- Architecture générale du système Git
- Un protocole de travail pour votre projet en utilisant GitLab

## Plan des activités:

- Etude d'un cas fictif mais inspiré d'histoires réelles: lecture et discussion (30 minutes)
- Cours Magistral: les systèmes de gestion de version
- Retour sur l'étude de cas
- Cours Magistral: description de la manière dont vous pourrez travailler (on fera une partie en labo).

## Après la séance vous devriez être capable de:

- Expliquer pourquoi utiliser un système de gestion de version est essentiel
- Lister les bonnes pratiques

## Le labo et quelques cours suivants reprendrons le sujet pour l'étendre (application dans le projet, culture DevOps)

## Note importante! Disclaimer!

### Ce que le cours ne peut pas faire

- La gestion de version est une spécialité à part entière en ingénierie de gestion de projet. Git est une technologie très flexible, mais son architecture peut paraître complexe.
- Il est impossible matériellement, à ce stade de votre scolarité et avec le temps que nous avons ici de vous donner des connaissances complètes. Cela vous prendra du temps. C'est normal.
- La présentation faite dans le cadre de ce cours utilise donc quelques raccourcis, qui vous seront signalés, et que vous dépasserez quand vous gagnerez en expérience.

## Note importante! Disclaimer!

### Nos objectifs:

- Comprendre l'importance de gérer les versions d'un projet en équipe
- Etre capable d'utiliser GitLab selon un workflow prescrit à l'avance pour votre projet
- Vous donner le vocabulaire qui vous permettra de progresser si nécessaire, ainsi que des références sur des outils ou ouvrages pertinents.
- Dans les cours ultérieurs comprendre comment tout cela s'intègre dans le cycle de vie logiciel (Intégration Continue, *DevOps*)

# Bulle et Bob font des sites web

# Qu'est-ce qu'un système de contrôle de versions

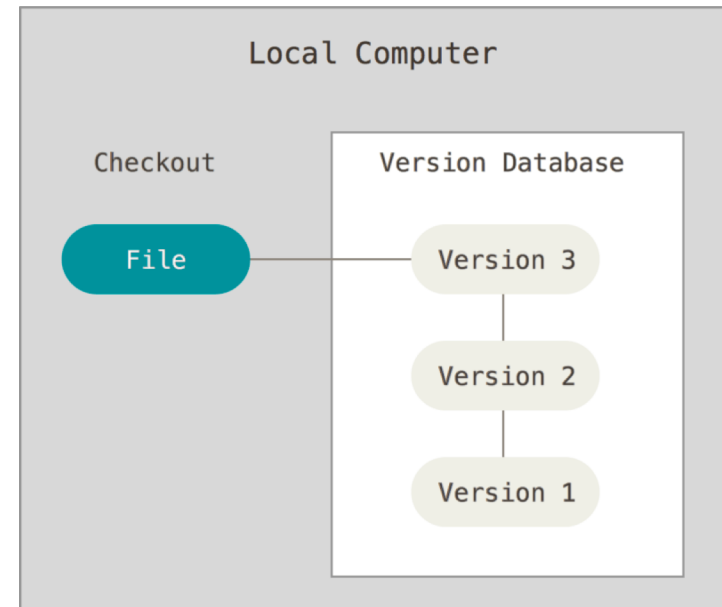
**(Source Control Management – SCM, Version Control System – VCS, Revision Control Management - RCM)**

- Système permettant d'archiver, de manipuler, de fusionner, les évolutions d'un projet dans le temps, indépendamment des langages et des plateformes.
- Permet d'organiser le développement et la maintenance du logiciel au sein d'une équipe.
- On peut ainsi contrôler pour tout l'historique du projet:
  - Qui a fait quoi
  - Pour quelle raison
  - Quand

**On peut revenir en arrière si on a fait des bêtises...**

# Gestion de version locale (comme RCS)

- Plutôt pour quand on travaille seul, ou a plusieurs sur un seul poste.
- Un peu plus structuré que d'organiser ses versions en répertoires.
- RCS Stocke en « patch » les différences entre deux versions pour pouvoir les restituer

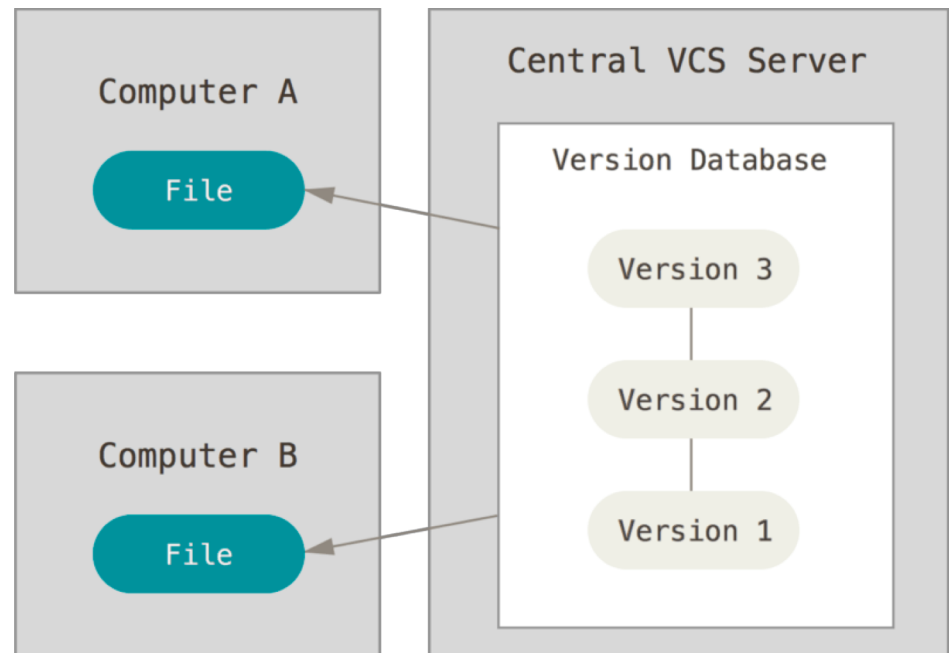


Source image: Pro Git, by Scott Chacon and Ben Straub



# Gestion de Version Centralisée (CVS, Subversion)

- Pour travailler à plusieurs, et organiser le développement en parallèle
- Stocke en « patch » les différences entre deux versions pour pouvoir les restituer
- Exemples: subversion et la plateforme de code open source sourceforge.
- Problème: *single point of failure*

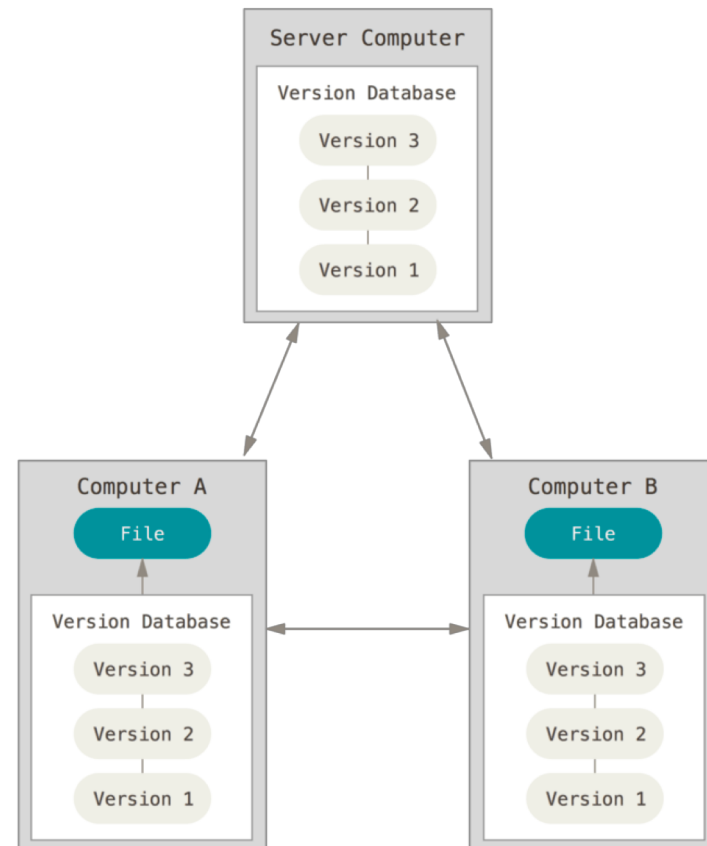


Source image: Pro Git, by Scott Chacon and Ben Straub



# Gestion de Version Distribuée: Git, Mercurial...

- Plus de single point of failure
- Plus de souplesse dans la définition du protocole de travail (*workflow*).
- Courbe d'apprentissage un peu plus raide au début



Source image: Pro Git, by Scott Chacon and Ben Straub

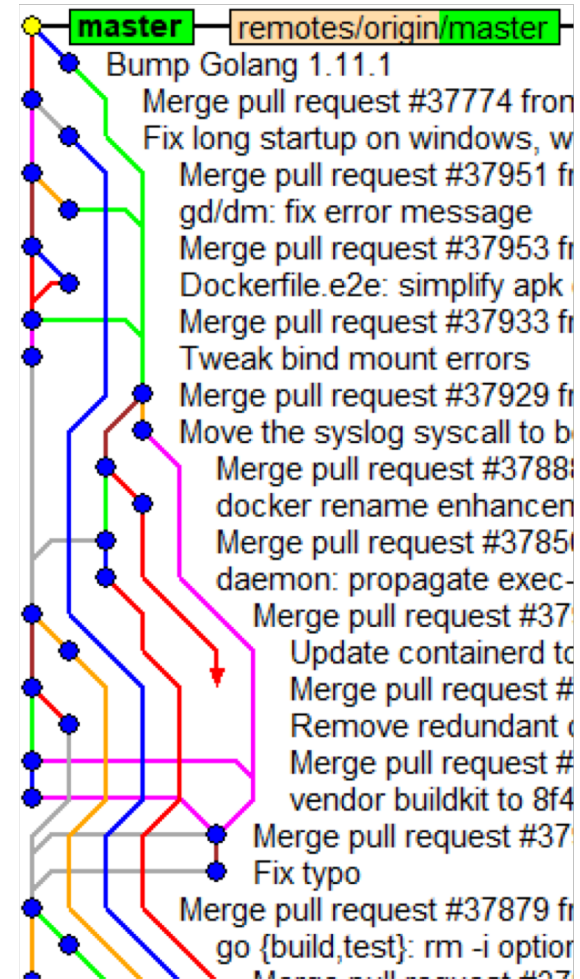


# Concepts Généraux Git

# Concepts et définitions pour gérer les versions avec Git

Un projet qu'on développe en utilisant Git peut se représenter sous forme d'un graphe reflétant son historique, dont les nœuds sont étiquetés, et contiennent chacun:

- le code
  - l'auteur
  - d'autres informations comme des commentaires
- d'une version donnée du projet.

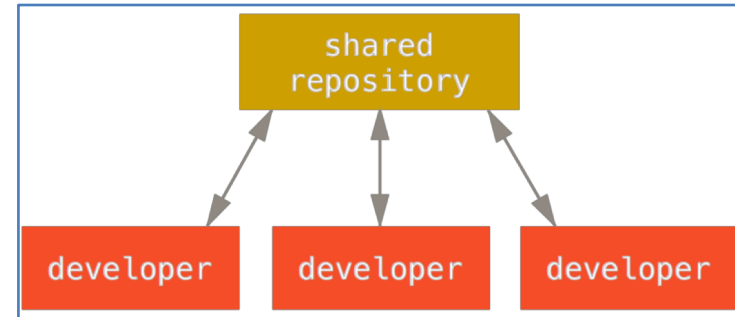


## Concepts et définitions pour gérer les versions avec Git

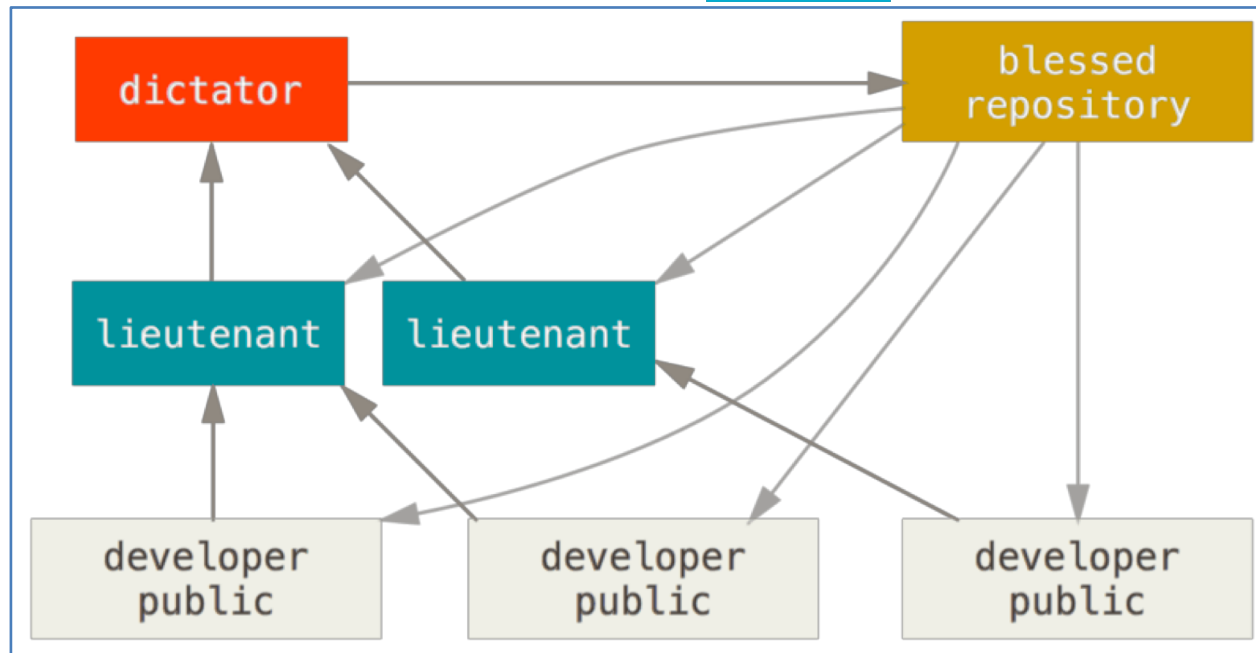
- **Dépôt** : une instance du projet et de son historique avec toutes les informations de gestion de version associées.
- Dépôts particuliers, quand on utilise GitLab ou GitHub par exemple:
  - **Dépôt de Référence**: le dépôt qui fait foi du projet
  - **Dépôt *Origine* (*upstream*)** : dépôt distant qui est copié (*clone*) en local pour travailler
  - **Dépôt *Fork*** : une copie distante du dépôt de référence.

## Notion de *Workflow*

- Git est un outil de gestion de version non prescriptif. Il faut définir un *Workflow*, un processus de travail, pour organiser les développements.



Source images: Pro Git, by Scott Chacon and Ben

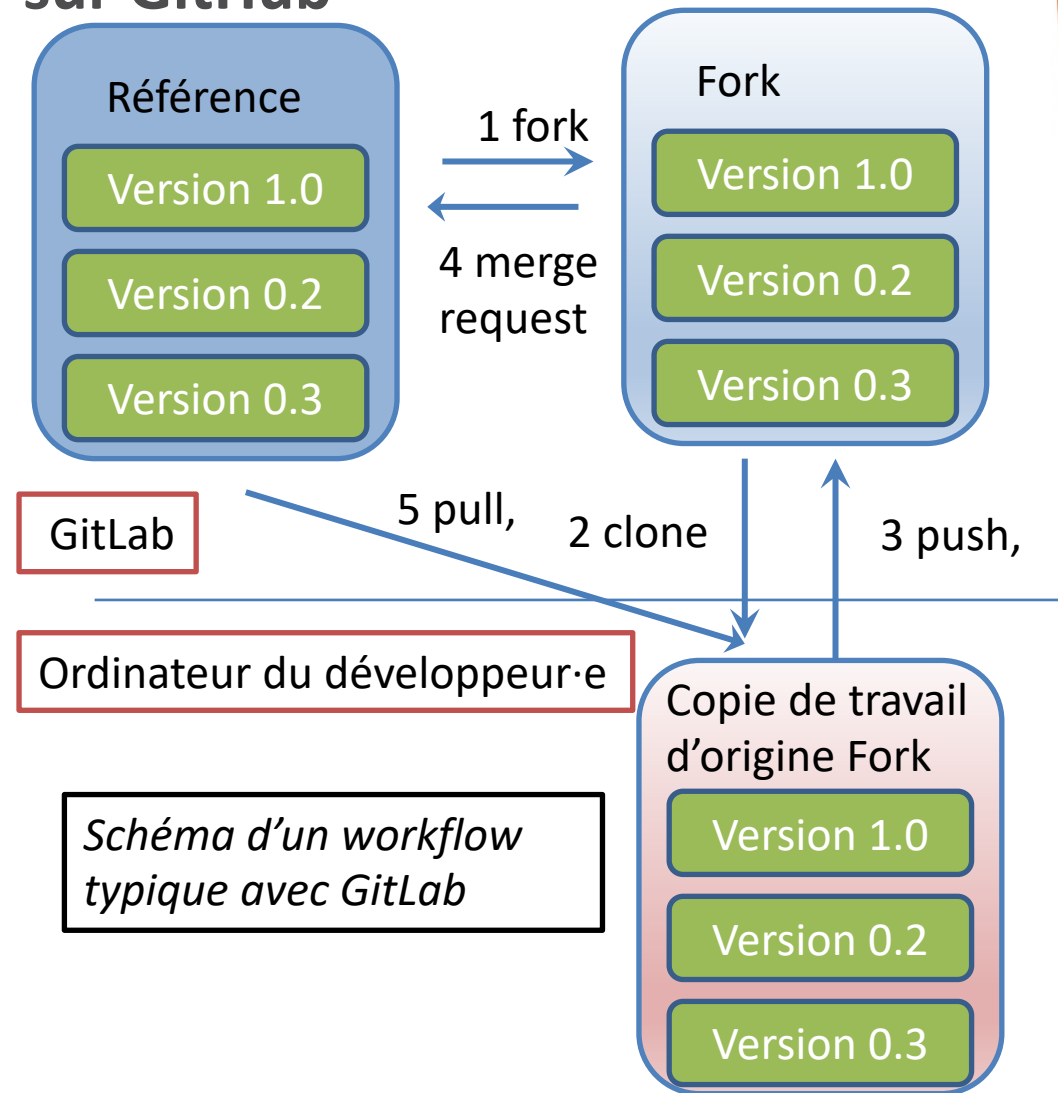


## Workflow préféré avec GitLab, pour contribuer sur GitHub

■ Quand on travaille en équipe, avec GitLab, on crée souvent une copie distante (**fork**).

■ Quand on a fini de travailler sur le dépôt *Fork*, on crée une *merge-request* (demande de fusion, se nomme *pull – request* sur Github).

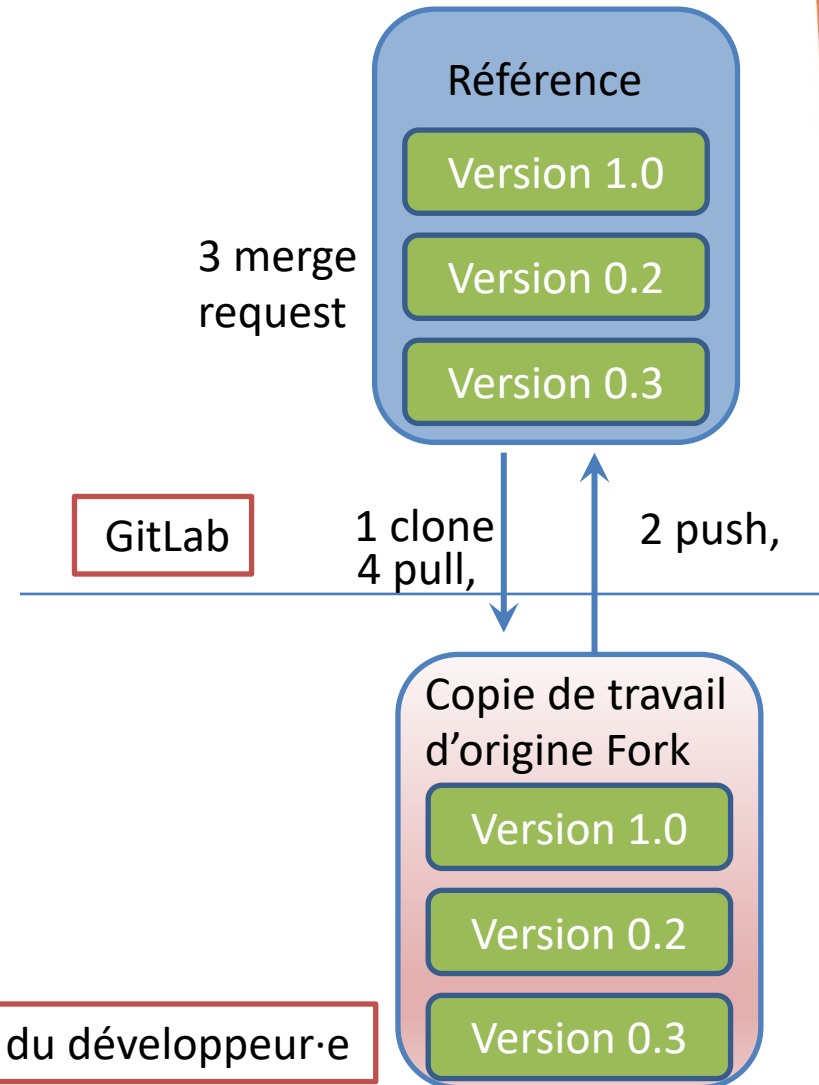
■ Un·e *mainteneur·e* accepte ou non de fusionner les changements.



## Workflow simplifié avec GitLab

■ Vous pouvez simplifier ce workflow pour votre projet en travaillant toujours sur le projet de référence

- Quand on a fini de travailler sur une branche en local, on push cette branche sur le projet Référence et on crée une merge-request.
- Un·e *mainteneur·e* accepte ou non de fusionner les changements.



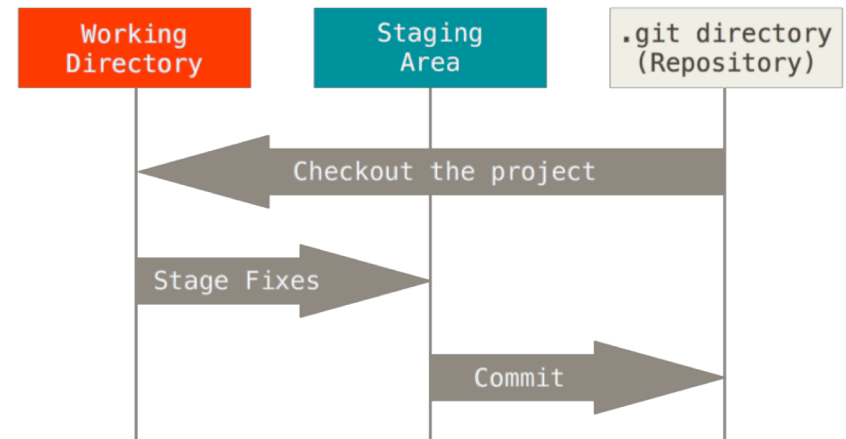
Ordinateur du développeur·e

# Créer un dépôt et travailler en local

# Architecture du dépôt Git

## 3 états des fichiers dans un dépôt local Git:

- **commité**: stocké dans la base de données locale, le répertoire git
- **modifié**: modifié, mais les changements n'ont pas encore été stockés dans la base de données locale
- **staged** le fichier modifié a été marqué pour faire partie du stockage dans la base de données locale lors du prochain commit.



Source image: Pro Git, by Scott Chacon and Ben Straub



## 3 zones correspondantes pour le dépôt local:

- **répertoire git**: crée lors d'un clone, toutes les données et meta-données du projet
- **répertoire de travail**: résultat d'une extraction d'une version du projet du répertoire git
- **zone de staging** fichier qui stocke les informations sur les prochains candidats au commit

# Créer un dépôt local

## ■ Créer un dépôt git à partir de vos sources

- se placer dans le répertoire du projet et saisir: **git init**

Crée un repertoire .git

- ajouter les fichiers à versionner: **git add nomfichier**

- enregistrer tout dans la base de données: **git commit**

## ■ Cloner un dépôt existant qui se trouve à l'adresse url

- se placer dans le répertoire où on veut placer le dépôt et taper: **git clone url rep**

Crée un répertoire **rep**, y met un repertoire **.git** avec toutes les données du dépôt distant, et extrait une copie de travail de la dernière version

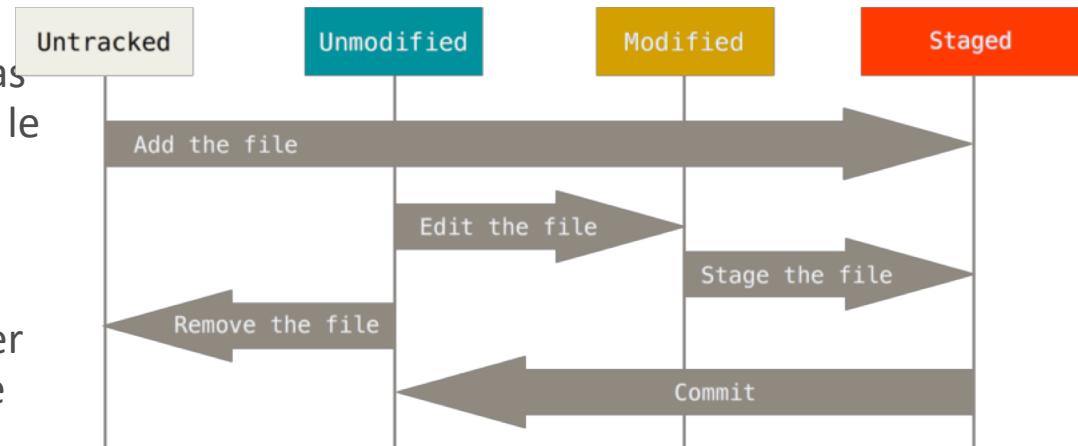
- selon le protocole utilisé, l'url pourra s'écrire de manière différente (git, ssh, https)

Exemple pour https (que vous pourrez utiliser avec GitLab):

**git clone https://github.com/jff/TeLLer.git**

# Etats possibles des fichiers

- Untracked: git ne les connaît pas.
- Unmodified: connu de git, mais pas modifié par le développeur depuis le dernier commit.
- Modified: vous avez modifié le fichier
- Staged: vous avez marqué le fichier modifié comme devant faire partie du prochain commit



Source image: Pro Git, by Scott Chacon and Ben Straub



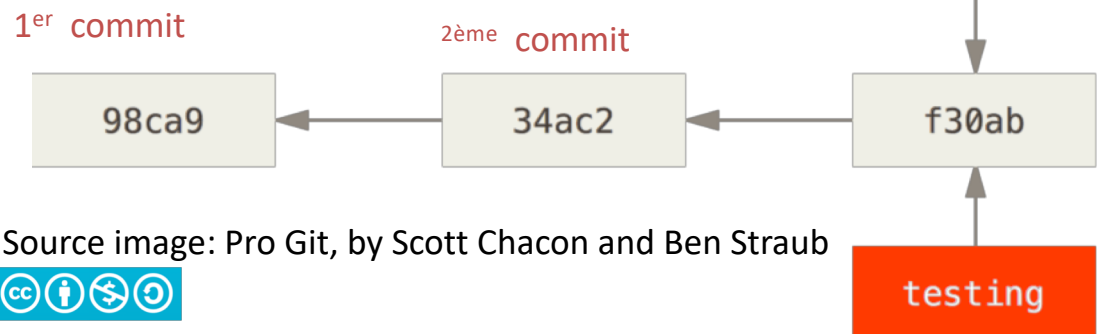
- Untracked -> Staged: **git add filename**
- Modified -> Staged: **git add filename**
- Unmodified -> Untracked: **git remove filename**
- (Modified-> Unmodified en annulant les modifications: **git checkout filename**)
- Staged -> Unmodified en sauvegardant les modifications dans la base locale: **git commit filename**
- pour une gestion interactive: **git add -p, git checkout -p**
- Pour annuler le dernier commit git **reset HEAD^**(sans remettre le contenu des fichiers en l'état dans la copie de travail sinon on ajoute **git reset -- hard** )

# Branches

Les branches permettent de créer une version disjointe du projet sur lesquelles on peut travailler en parallèle

- Une branche git est une étiquette vers un commit
- La branche originelle est nommée master
- Une étiquette **HEAD** désigne la branche sur laquelle on travaille
- On peut créer une nouvelle branche de nom « testing » en utilisant la commande:

■ **git branch testing**



Source image: Pro Git, by Scott Chacon and Ben Straub

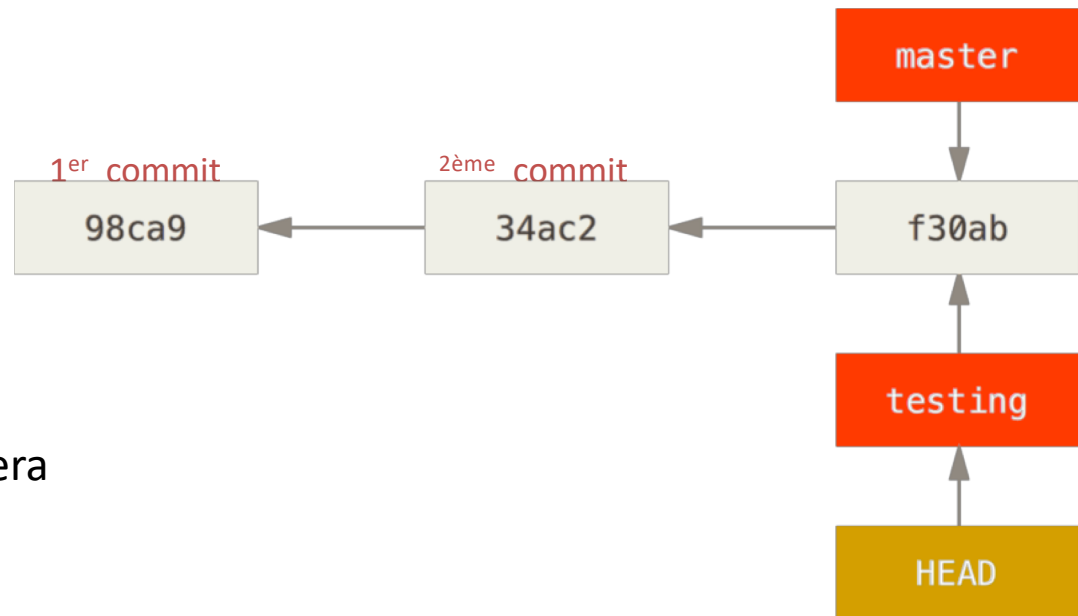


## Branches

■ On peut se déplacer sur une branche **testing** pour y travailler en utilisant la commande:

■ **git checkout testing**

■ L'étiquette HEAD est maintenant sur la nouvelle branche



Souvent on créera et se placera sur une nouvelle branche en une ligne avec:

**git checkout -b testing**

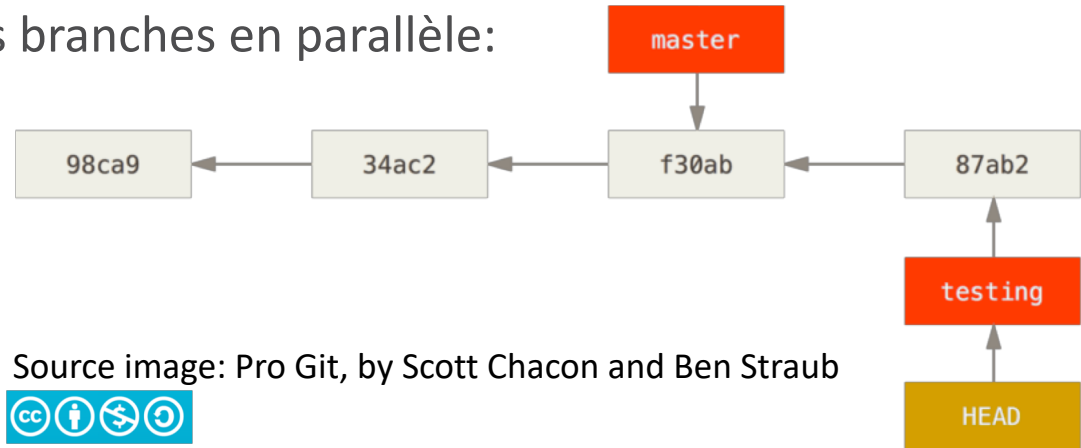
Source image: Pro Git, by Scott Chacon and Ben Straub



# Branches

On peut alors travailler sur les branches en parallèle:

git commit -am 'j'ai changé'

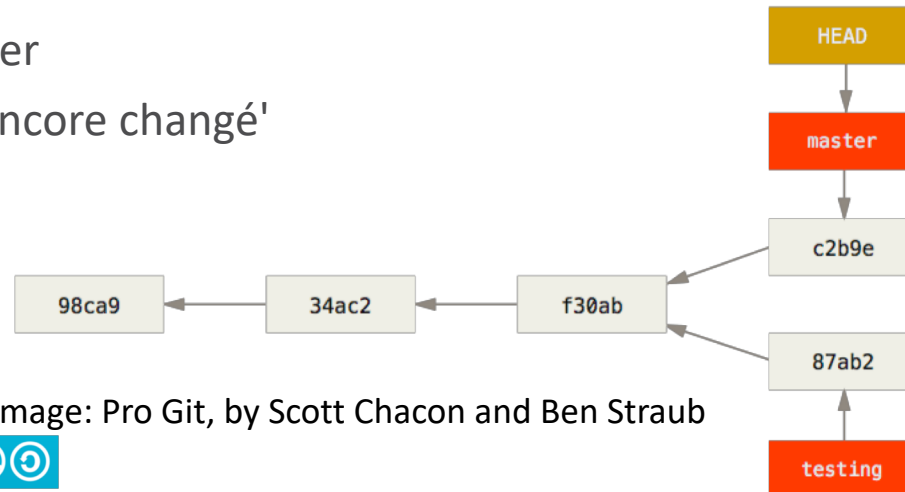


Source image: Pro Git, by Scott Chacon and Ben Straub



git checkout master

git commit -am 'encore changé'

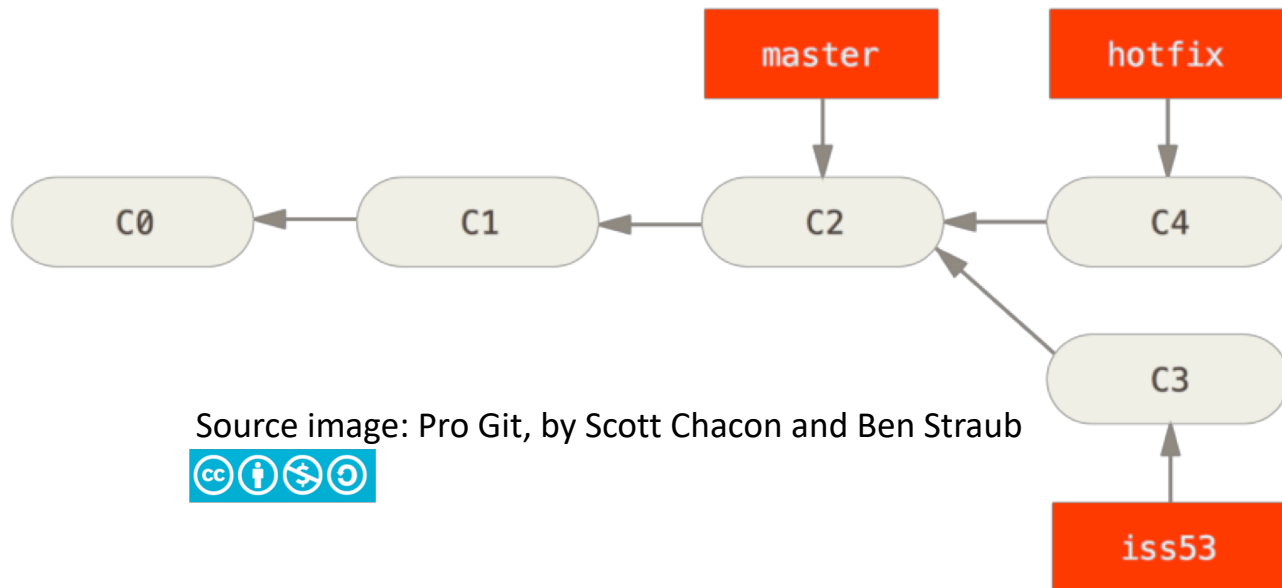


Source image: Pro Git, by Scott Chacon and Ben Straub



# Fusionner des branches

- Quand on a fini, on peut décider de fusionner les branches. Selon les modifications apportées, les techniques utilisées par git seront différentes.
- Imaginons qu'on travaille sur une évolution de fonctionnalité iss53 mais qu'on a du faire une correction de bug sur une branche hotfix en parallèle

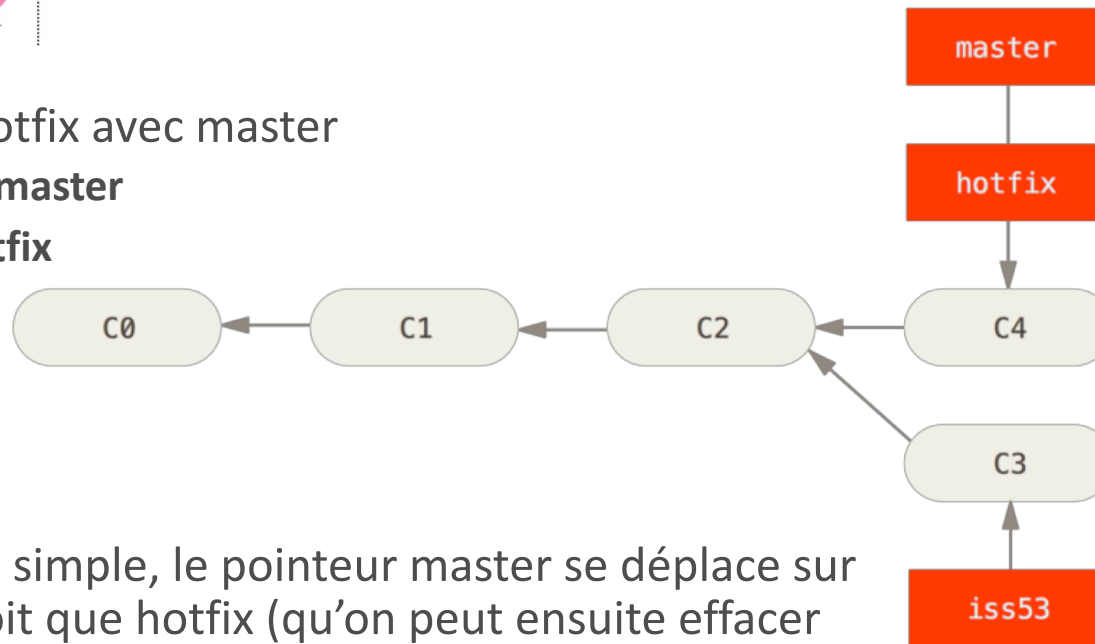


# Fusionner des branches

On fusionne hotfix avec master

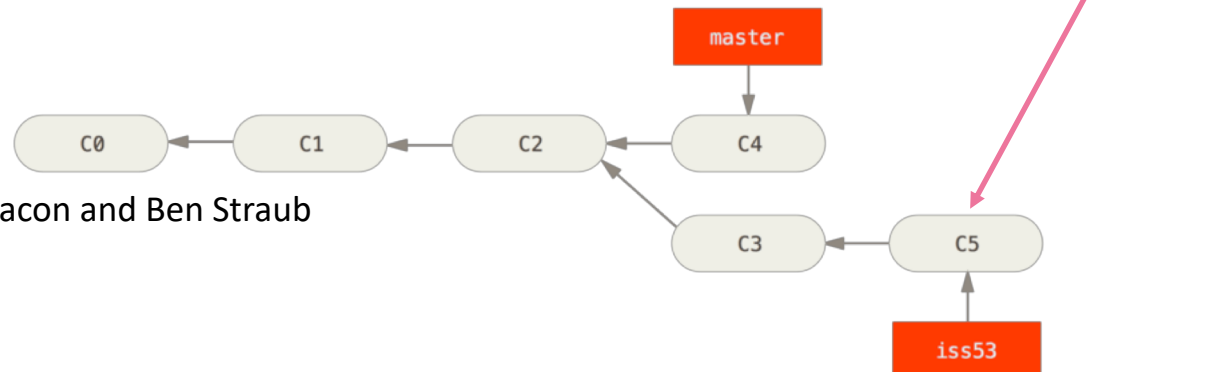
git checkout master

git merge hotfix



Ici le merge est simple, le pointeur master se déplace sur le même endroit que hotfix (qu'on peut ensuite effacer avec:

git branch -d hotfix



Source image: Pro Git, by Scott Chacon and Ben Straub



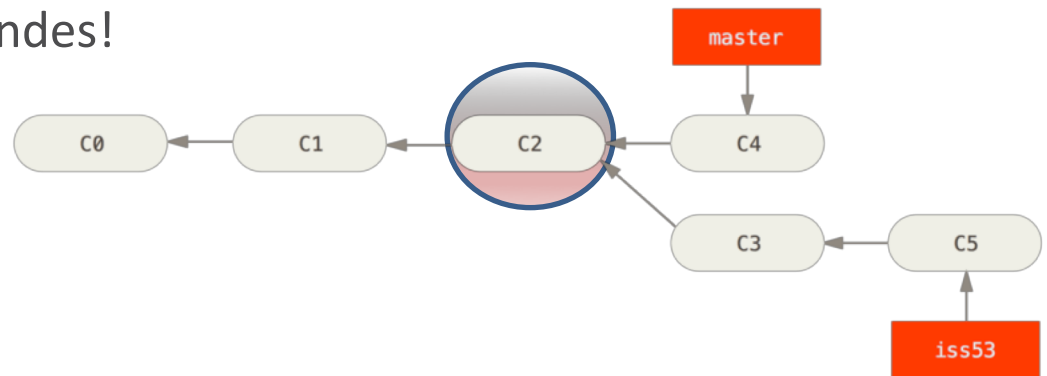
# Fusionner des branches

Le second merge est plus complexe: on a maintenant deux versions divergentes avec un ancêtre commun.

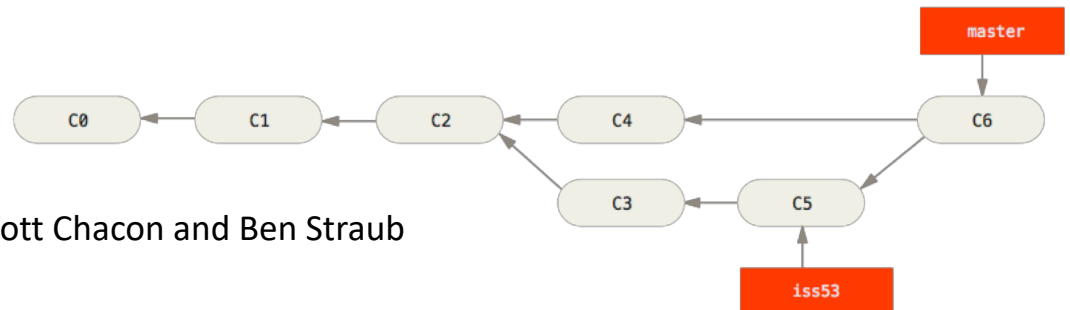
On utilise les mêmes commandes!

**git checkout master**

**git merge iss53**



Si tout va bien (les modifications ne sont pas exactement sur le même code) git se débrouille et crée un nouvel instantané du projet qui correspond à la fusion.



Source image: Pro Git, by Scott Chacon and Ben Straub

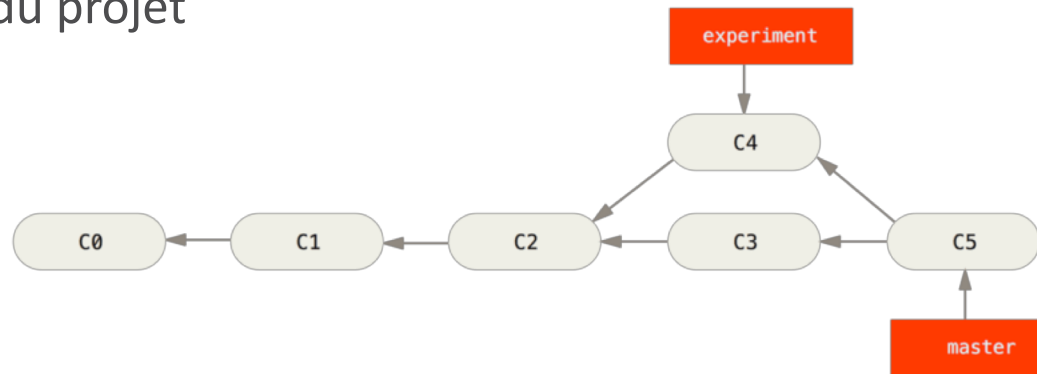


# Problèmes après la fusion

- Git peut échouer à faire la fusion dans le cas précédent.
  - Si les modifications portent sur le même code
  - Un fichier supprimé dans une branche et modifié dans l'autre
  - ...
  
- La fusion peut aboutir à un conflit que faire:
  - Résoudre manuellement en indiquant quel code privilégier ou éditer le code à la main
  - Faire un commit
- Pour annuler un commit, on fait:
  - git reset

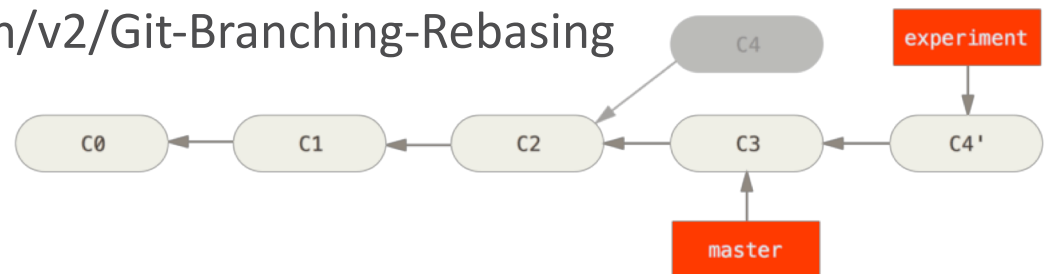
## Rebase – pour plus tard

■ *merge* permet de fusionner les commits en gardant l'historique des instantanés du projet



■ *rebase* applique les commits à une nouvelle base! (on ne va pas trop jouer avec ça au début, ça réécrit l'historique du projet)

<https://git-scm.com/book/en/v2/Git-Branching-Rebasing>



Source images: Pro Git, by Scott Chacon and Ben



## Outils autour de Git

- GitLab: plus que Git, c'est un ensemble d'outils soutenant le développement logiciel dans une culture *DevOps*. Vous utiliserez une petite partie de ses fonctionnalités pour le projet. On regardera le reste dans un cours ultérieur pour illustrer.
- GitHub: en ligne, partage gratuit de dépôts open source. Orienté logiciel libre et collaboration. Récemment racheté par Microsoft.
- Clients graphiques git, outils de visualisation de dépôt: utilisables en local ces outils vous permettront de manipuler et visualiser le graphe des versions du projet, et où en sont les branches.
  - gitk (vient avec git quand on l'installe)
  - TortoiseGit pour windows
  - ...
- Plugin intégrés dans les environnements de développement.
  - Netbeans: onglet Team
  - Eclipse: Egit

# Un protocole de travail sous GitLab pour votre projet

# Mettre en place un projet sous GitLab

- Un·e des membres du groupe projet crée un Groupe.
  - <https://docs.gitlab.com/ee/user/group/>
- Il·elle ajoute les autres membres du groupe et leur donne des droits
  - <https://docs.gitlab.com/ee/user/permissions.html>
  - **Owner > maintenir > developer > reporter > guest**
  - Si on est pas au moins **developer** on ne peut pas faire grand chose

# Mettre en place un projet sous GitLab

## ■ Créer un projet **dans le Groupe**. Deux cas possibles:

- Il n'y a pas encore de code: initialisez le avec un readme dans l'interface de création.
- Il y a déjà du code sur votre machine. En local:
  - créer un projet git à la racine de votre code source.
    - ▶ **git init**
  - (paramétrez votre projet avec vos identifiants enib, ou vérifiez que c'est fait correctement avec **git config --list**:
    - ▶ **git config --global user.name "bosser"**
    - ▶ **git config --global user.email "anne-gwenn.bosser@enib.fr" )**
  - spécifier l'origine distante du projet sur GitLab (le nom en local sera origin):
    - ▶ **git remote add origin https://git.enib.fr/mygroup-cpos6/projetDeGroupe.git**
  - tracker tous les fichiers et sous fichiers du répertoire:
    - ▶ **git add .**

# Mettre en place un projet sous GitLab

- On peut entre chaque commande utiliser **git status** pour voir ce qu'il se passe
- Commiter les fichiers
  - ▶ **git commit -m "Initial commit"**
- « Pousser » les fichiers sur la branche distante
  - ▶ **git push -u origin master**
  - <https://git-scm.com/docs/git-push>
  - Poste sur l'origin, sur la branche master, le contenu de la branche master locale.

# Facultatif pour chaque developpeur·e: créer un fork dans votre namespace

1 – Créer votre Fork

projetDeGroupe Private Add license

Project ID: 246

0 Star 0 Fork HTTPS https://git.enib.fr/g

Files (113 KB) Commit (1) Branch (1) Tags (0) Sec

Add Readme Add Changelog Add Contribu

master projetDeGroupe / +

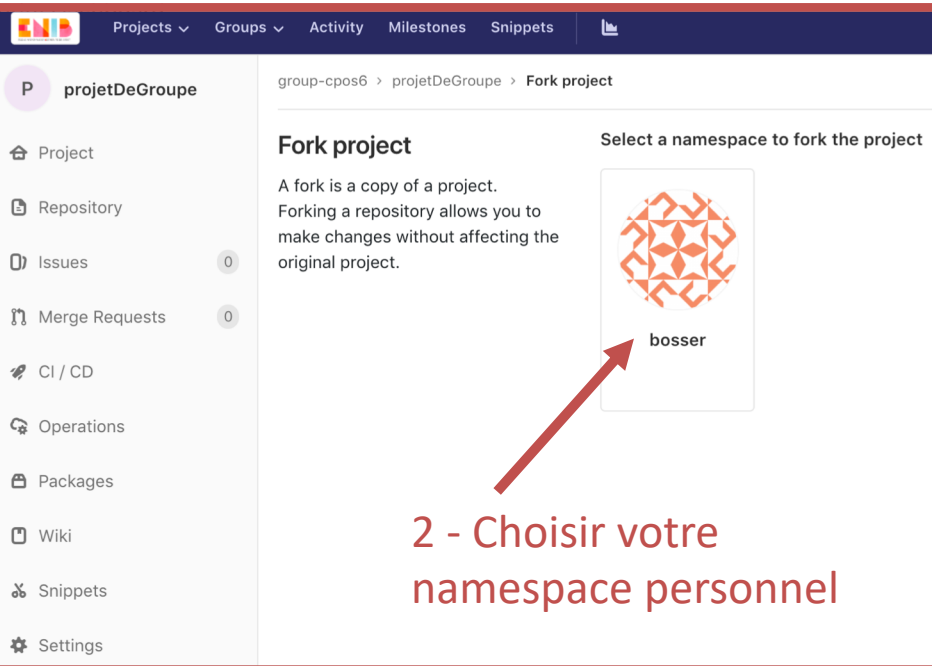


commit initial  
bosser authored 8 minutes ago

Name

.DS\_Store

classes.py



group-cpos6 > projetDeGroupe > Fork project

### Fork project

A fork is a copy of a project. Forking a repository allows you to make changes without affecting the original project.

Select a namespace to fork the project

bosser

2 - Choisir votre namespace personnel

## Workflow pour chaque membre de l'équipe: travail en local (1)

### 1 - Cloner:

■ **git clone** <https://git.enib.fr/bosser/projetDeGroupe.git>

### 2- Avant de commencer à faire des modifications, créer une branche et s'y déplacer:

■ **git branch branchDeModification**

■ **git checkout branchDeModification**

### 3- Stage des modifications

■ **git add nomfichier**, ou **git add -p**

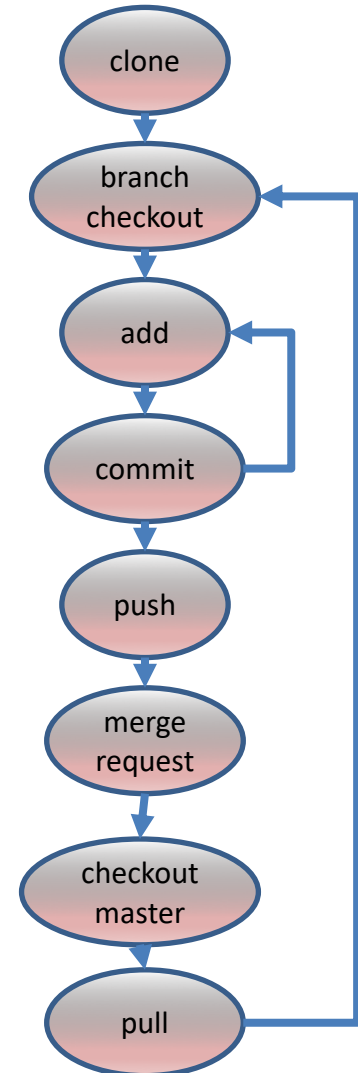
### 4- Commiter les fichiers staged

■ **git commit**

### 5- Reprendre les étapes 3 et 4 autant que nécessaire

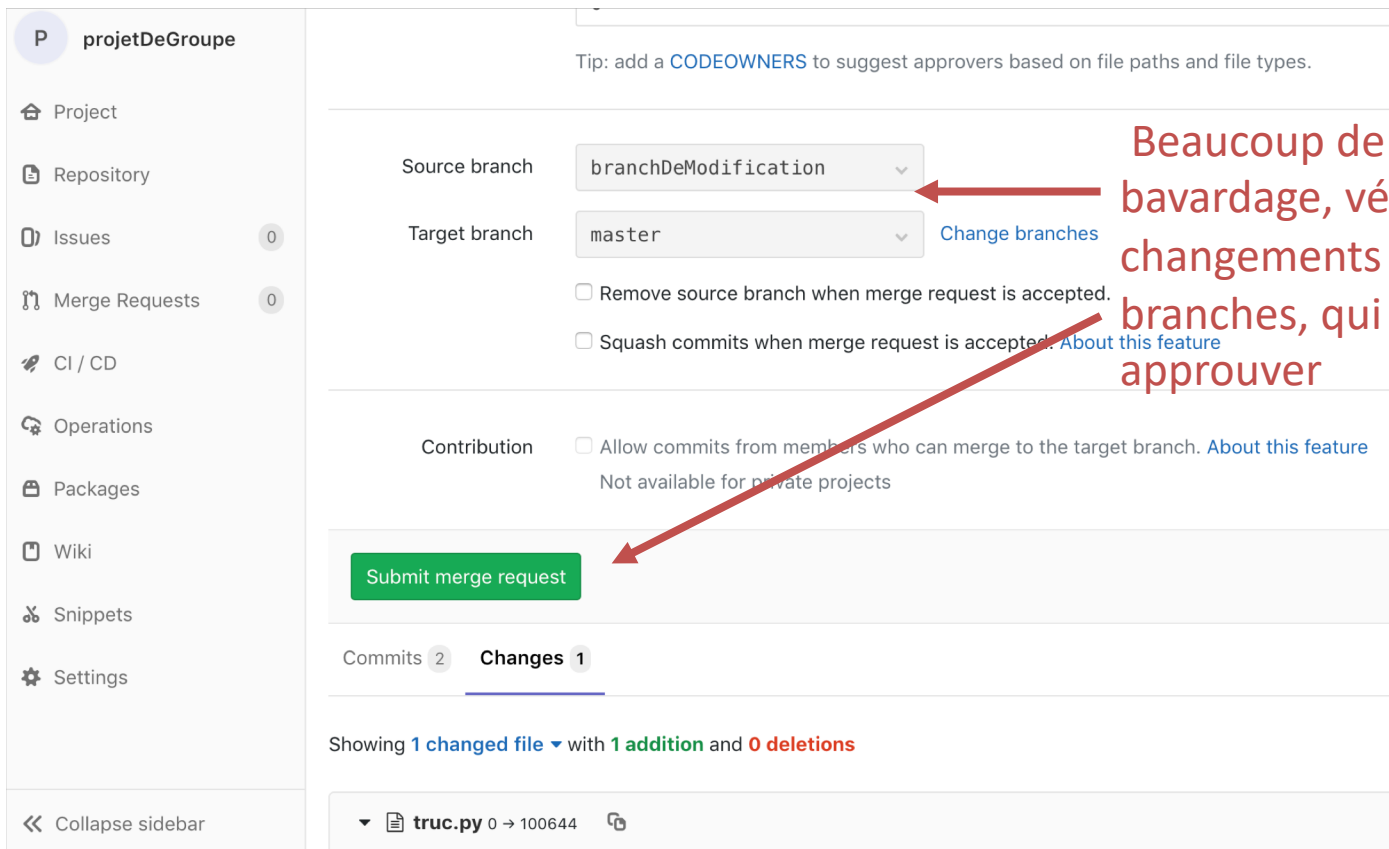
### 6- Pousser les modifications sur l'origine

■ **git push origin branchDeModification**



## Workflow pour chaque membre de l'équipe: travail en local (2)

Le résultat du push vous proposera alors une url pour faire une merge-request. Faites la en copiant l'url dans un navigateur



The screenshot shows the GitLab merge request interface for a project named "projetDeGroupe". The left sidebar contains navigation links: Project, Repository, Issues (0), Merge Requests (0), CI / CD, Operations, Packages, Wiki, Snippets, and Settings. The main content area has a tip: "Tip: add a [CODEOWNERS](#) to suggest approvers based on file paths and file types." Below this, the "Source branch" is set to "branchDeModification" and the "Target branch" is set to "master". There are two checkboxes: "Remove source branch when merge request is accepted." and "Squash commits when merge request is accepted.", both of which are unchecked. A link "Change branches" is next to the target branch dropdown. Under the "Contribution" section, there is a checkbox "Allow commits from members who can merge to the target branch." which is unchecked, with a link "About this feature" and the text "Not available for private projects". A green button labeled "Submit merge request" is prominently displayed. Below the button, there are tabs for "Commits 2" and "Changes 1", with "Changes 1" being the active tab. The changes section shows "Showing 1 changed file" with a dropdown arrow, and details "with 1 addition and 0 deletions". At the bottom, a file named "truc.py" is shown with a version change from 0 to 100644. Two red arrows point from the text on the right to the "branchDeModification" dropdown and the "Submit merge request" button.

Tip: add a [CODEOWNERS](#) to suggest approvers based on file paths and file types.

Source branch: [branchDeModification](#)

Target branch: [master](#) [Change branches](#)

☐ Remove source branch when merge request is accepted.

☐ Squash commits when merge request is accepted. [About this feature](#)

Contribution ☐ Allow commits from members who can merge to the target branch. [About this feature](#)  
Not available for private projects

[Submit merge request](#)

Commits 2 **Changes 1**

Showing 1 changed file ▾ with 1 addition and 0 deletions

▼ **truc.py** 0 → 100644

Beaucoup de bavardage, vérifier les changements et les branches, qui doit approuver

## Workflow pour chaque membre de l'équipe: travail en local (3)

- Se remettre sur la branche master et faire une mise à jour (pour récupérer le code des autres par exemple):

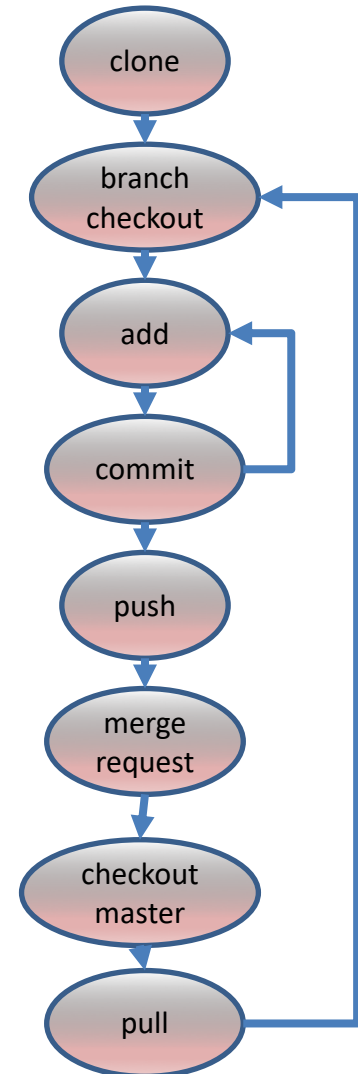
- `git checkout master`

- `git pull reference master` (ou `git pull` dans le cas où on travaille sur le fork)

- Reprendre à l'étape 2: créer une nouvelle branche pour la nouvelle modification

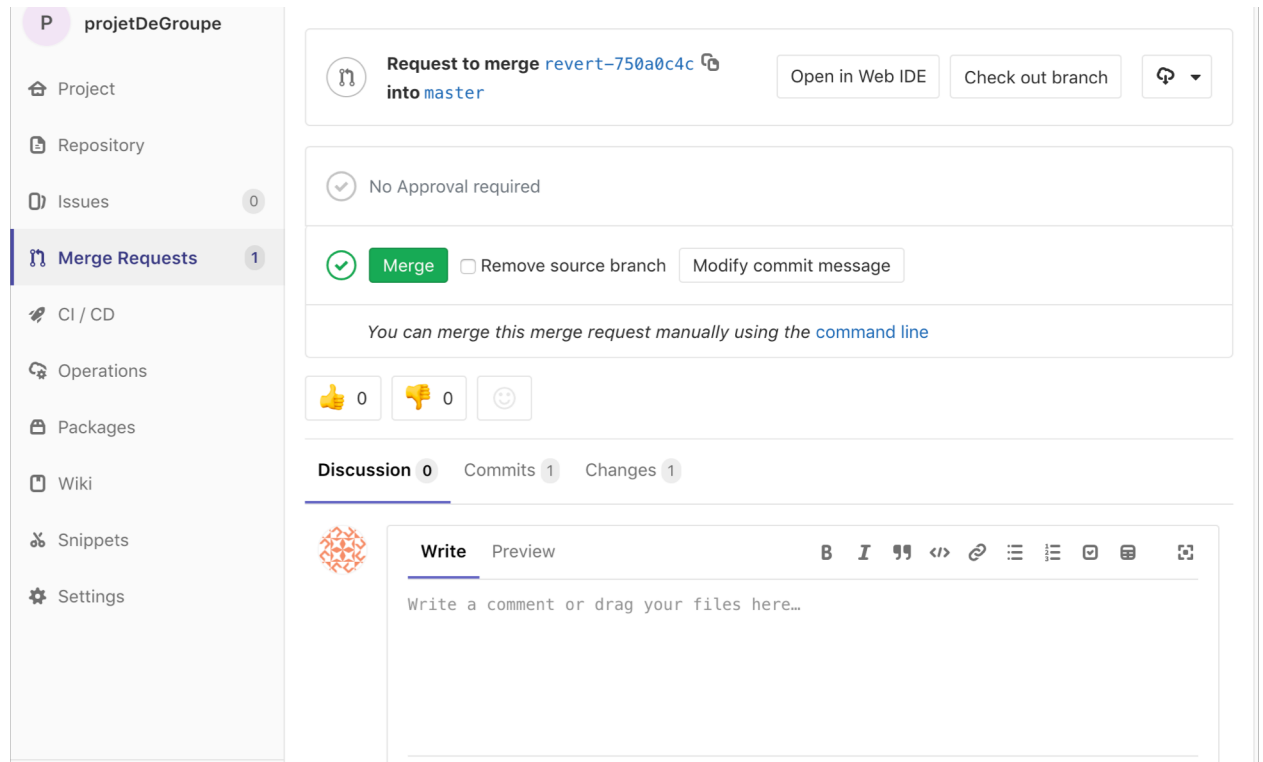
Dans le cas du workflow préféré on peut avoir besoin d'ajouter un hôte distant en faisant `git remote add reference url`

- `git log` vous permet de regarder l'historique des commit



# Pour le/la mainteneur·e: traiter une merge request (1)

- Le projet peut définir des membres qui doivent approuver des modifications mais par défaut un·e mainteneur·e effectue le merge sur GitLab de la branche du fork sur la branche master du projet référence



The screenshot shows the GitLab Merge Request interface for a project named "projetDeGroupe". The left sidebar contains navigation links: Project, Repository, Issues (0), Merge Requests (1), CI / CD, Operations, Packages, Wiki, Snippets, and Settings. The main content area displays the merge request details. At the top, it says "Request to merge revert-750a0c4c into master" with buttons for "Open in Web IDE", "Check out branch", and a dropdown menu. Below this, it indicates "No Approval required". A green "Merge" button is visible, along with a checkbox for "Remove source branch" and a "Modify commit message" button. A message states: "You can merge this merge request manually using the [command line](#)". There are three reaction buttons: thumbs up (0), thumbs down (0), and a smiley face. At the bottom, there are tabs for "Discussion" (0), "Commits" (1), and "Changes" (1). The "Discussion" tab is active, showing a "Write" button, a "Preview" tab, and a rich text editor with the placeholder text "Write a comment or drag your files here...".

## Pour le/la mainteneur·e: traiter une merge request (2)

- On conseille de créer une nouvelle branche sur sa copie locale et de tester la fusion (le/la mainteneur·e est souvent aussi développeur).
- Par exemple dans le cas de notre Workflow simplifié on pourra faire
  - `git checkout master`: se replacer sur la branche master depuis la branche de travail
  - `git pull origin master`: met à jour la branche master locale
  - `git checkout -b testMerge`: création d'une branche pour tester la fusion
  - `git pull origin nomBrancheAFusionner`
  - On teste:
    - Si tout va bien, on retourne sur GitLab pour accepter la fusion
    - Sinon, on fait des retours, ou on la refuse
  - `git checkout master`: on se replace sur master ou sur une autre branche de travail
  - `git branch -D testMerge`: on efface la branche

## Pour Le/la mainteneur·e: traiter une merge request

### ■ Gestion des conflits:

- on peut utiliser un outil de merge pour comparer les versions et voir ce qui ne fonctionne pas bien.
- on utilise **git reset HEAD^** pour annuler le dernier commit (en local pour annuler un commit **git reset -hard** permet de remettre les fichiers à la version précédente dans l'espace de travail dans le même temps)

## Commandes pratiques (les tester, rechercher leur documentation)

- `git status`
- `git log -- graph`
- `git config --list`
- `git remote add reference url`
  
- La commande magique **rebase** peut être utile quand on a besoin de modifier un commit passé:
  - remanier la branche de travail pour pouvoir faire une mise à jour « propre »: correction de typo sur tout l'historiques, commentaires, ...
  - une merge-request que vous avez soumise prend un peu de temps à être intégrée, et entre temps le master a évolué, et la fusion créera des conflits: vous pouvez faire un pull du master avec rebase (**`git pull -r origin master`**) depuis votre branche). Cela vous permettra de travailler sur le conflit en local.

- ProGit, Scott Chacon, disponible gratuitement, sous licence Creative Commons, y compris en Français.
- <http://think-like-a-git.net/>



<https://xkcd.com/1597/>

|   | COMMENT                            | DATE         |
|---|------------------------------------|--------------|
| ○ | CREATED MAIN LOOP & TIMING CONTROL | 14 HOURS AGO |
| ○ | ENABLED CONFIG FILE PARSING        | 9 HOURS AGO  |
| ○ | MISC BUGFIXES                      | 5 HOURS AGO  |
| ○ | CODE ADDITIONS/EDITS               | 4 HOURS AGO  |
| ○ | MORE CODE                          | 4 HOURS AGO  |
| ○ | HERE HAVE CODE                     | 4 HOURS AGO  |
| ○ | AAAAAAA                            | 3 HOURS AGO  |
| ○ | ADKFJSLKDFJSDKLFJ                  | 3 HOURS AGO  |
| ○ | MY HANDS ARE TYPING WORDS          | 2 HOURS AGO  |
| ○ | HAAAAAAAANDS                       | 2 HOURS AGO  |

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

<https://xkcd.com/1296/>